

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Нейронные сети»
Тема: Классификация обзоров фильмов

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Классификация последовательностей - это проблема прогнозирующего моделирования, когда у вас есть некоторая последовательность входных данных в пространстве или времени, и задача состоит в том, чтобы предсказать категорию для последовательности. Проблема усложняется тем, что последовательности могут различаться по длине, состоять из очень большого словарного запаса входных символов и могут потребовать от модели изучения долгосрочного контекста или зависимостей между символами во входной последовательности. В данной лабораторной работе также будет использоваться датасет IMDb, однако обучение будет проводиться с помощью рекуррентной нейронной сети.

Задание.

- Ознакомиться с рекуррентными нейронными сетями
- Изучить способы классификации текста
- Ознакомиться с ансамблированием сетей
- Построить ансамбль сетей, который позволит получать точность не менее 97%

Выполнение работы.

Данные и предобработка

Исходные данные загружены из встроенного в Keras набора IMDb с ограничением словаря до 10 000 наиболее частотных слов (рисунок 1). После объединения обучающей и тестовой выборок (по 25 000 отзывов каждая) получен массив из 50 000 отзывов. Для обучения и тестирования данные были случайным образом разделены в соотношении 80/20 (40 000 обучающих, 10 000 тестовых).

Каждый отзыв представляет собой последовательность индексов слов. Для приведения всех последовательностей к единой длине использовано дополнение (padding) до 500 слов (с обрезанием более длинных и заполнением

нулями более коротких). Такая длина была выбрана эмпирически – средняя длина отзыва составляет около 235 слов, 500 слов покрывает подавляющее большинство рецензий.

Для всех моделей использовался слой Embedding, преобразующий целочисленные индексы слов в плотные векторы размерности 32. Это позволяет сети извлекать смысловые связи между словами.

```

▼ Загрузка данных

[3]
✓ 0s
def load_data(num_words=10000):
    (train_data, train_targets), (test_data, test_targets) = imdb.load_data(num_words=num_words)

    data = np.concatenate((train_data, test_data), axis=0)
    targets = np.concatenate((train_targets, test_targets), axis=0)

    return data, targets

[4]
✓ 6s
num_words = 10000
max_review_length = 500
embedding_dim = 32

data, targets = load_data(num_words=num_words)
word_index = imdb.get_word_index()

▼
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 — 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb\_word\_index.json
1641221/1641221 — 1s 0us/step

▼ Предобработка данных

[5]
✓ 1s
X_train, X_test, y_train, y_test = train_test_split(data, targets, test_size=0.2, random_state=42)
print(f"Размер обучающей выборки: {len(X_train)}")
print(f"Размер тестовой выборки: {len(X_test)}")

X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
print(f"Форма обучающих данных после паддинга: {X_train.shape}")
print(f"Форма тестовых данных после паддинга: {X_test.shape}")

▼
Размер обучающей выборки: 40000
Размер тестовой выборки: 10000
Форма обучающих данных после паддинга: (40000, 500)
Форма тестовых данных после паддинга: (10000, 500)

```

Рисунок 1. Загрузка и предобработка данных.

Модели и их обучение

1. Простая LSTM

Архитектура:

- Embedding (10000 входных слов, 32 размерность вектора, длина последовательности 500)
- LSTM (100 нейронов, dropout=0.2, recurrent_dropout=0.2)
- Плотный слой (1 нейрон, сигмоид)

Обучение проводилось на 5 эпохах с размером пакета 64, оптимизатором Adam, функцией потерь бинарная кросс-энтропия. Валидация проводилась на тестовой выборке. Результаты в ходе обучения представлены на рисунке 2.

Точность на тесте составила 0.8736, а время обучения ~787–672 секунд на эпоху.

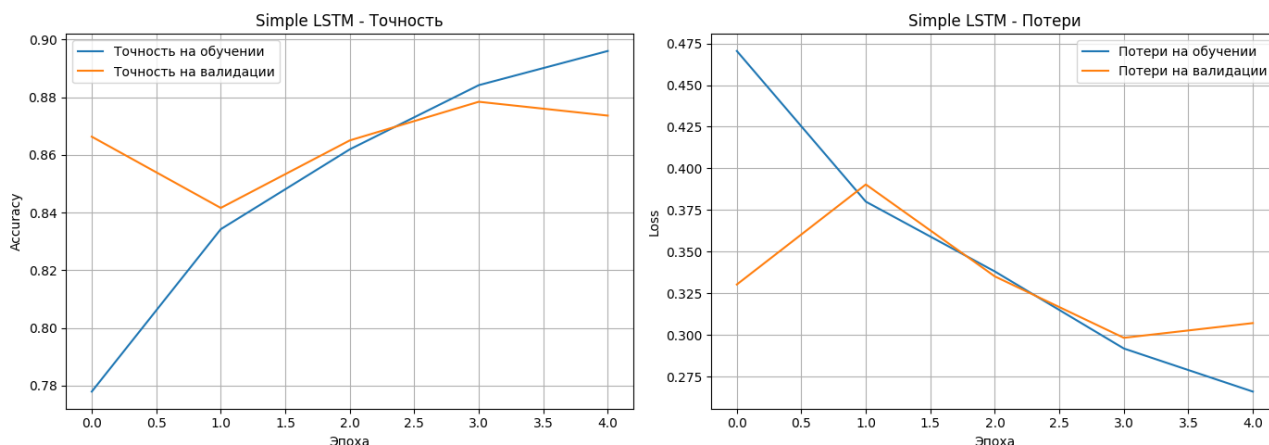


Рисунок 2. Точность и потери в ходе обучения простой LSTM модели.

По графикам можно видеть, что точность на обучении и валидации растет, но уже к 5 эпохе наблюдается небольшое расхождение (переобучение). Простая LSTM демонстрирует хорошие результаты, но не оптимальна из-за склонности к переобучению.

2. LSTM с Dropout

Архитектура:

- Embedding (10000, 32, input_length=500)
- Dropout (0.5)
- LSTM (128 нейронов, dropout=0.3, recurrent_dropout=0.3)
- Dropout (0.5)
- Dense (1, сигмоид)

Идея заключается в добавлении двух слоев Dropout (после Embedding и после LSTM) должно уменьшить переобучение за счет случайного отключения части нейронов во время обучения.

Обучение проводилось на 5 эпохах с размером пакета 64, оптимизатором Adam, функцией потерь бинарная кросс-энтропия. Валидация проводилась на тестовой выборке. Результаты в ходе обучения представлены на рисунке 3.

Точность на тесте составила 0.8617, а время обучения ~854–902 секунд на эпоху (несколько больше, чем у простой LSTM, из-за большего количества параметров и дополнительных операций).

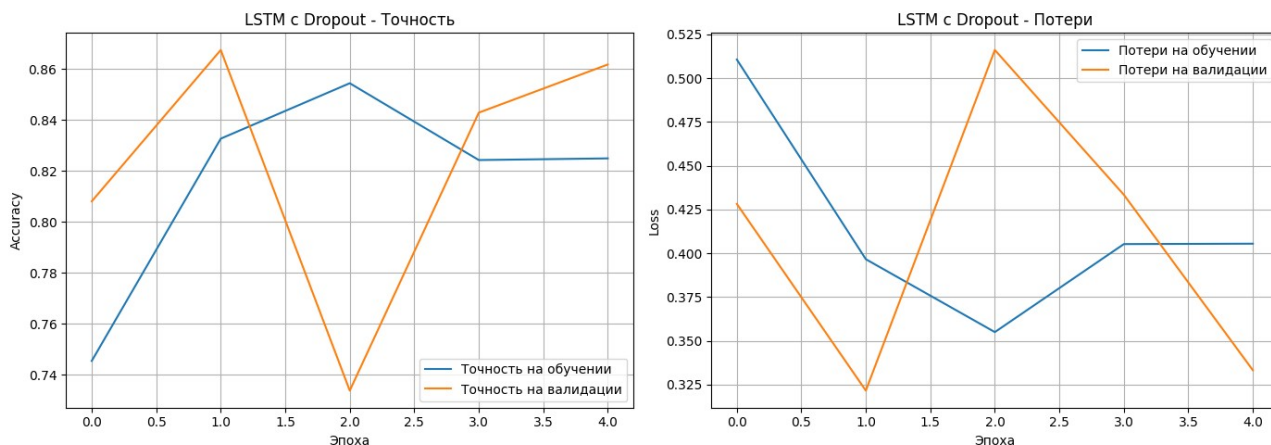


Рисунок 3. Точность и потери в ходе обучения LSTM с Dropout.

На графиках видно, что процесс обучения нестабилен: точность на валидации колеблется. Это может быть связано с тем, что слишком высокий dropout (0.5) препятствует эффективному обучению, или модель требует большего числа эпох. Несмотря на добавленные регуляризаторы, точность оказалась ниже, чем у простой LSTM.

3. Комбинированная модель CNN + LSTM

Архитектура:

- Embedding (10000, 32, input_length=500)
- Conv1D (32 фильтра, kernel_size=3, padding='same', activation='relu')
- MaxPooling1D (pool_size=2)
- LSTM (100 нейронов, dropout=0.2, recurrent_dropout=0.2)
- Dense (1, сигмоид)

Идея заключается в том, что сверточные слои выделяют локальные паттерны в последовательностях слов (например, биграммы, триграммы), которые затем обрабатываются LSTM, учитывающим долгосрочные зависимости.

Обучение проводилось на 5 эпохах с размером пакета 64, оптимизатором Adam, функцией потерь бинарная кросс-энтропия. Валидация проводилась на тестовой выборке. Результаты в ходе обучения представлены на рисунке 4.

Точность на тесте составила 0.8875, а время обучения ~339–386 секунд на эпоху (быстрее, чем предыдущие модели, благодаря меньшему числу параметров после пулинга).

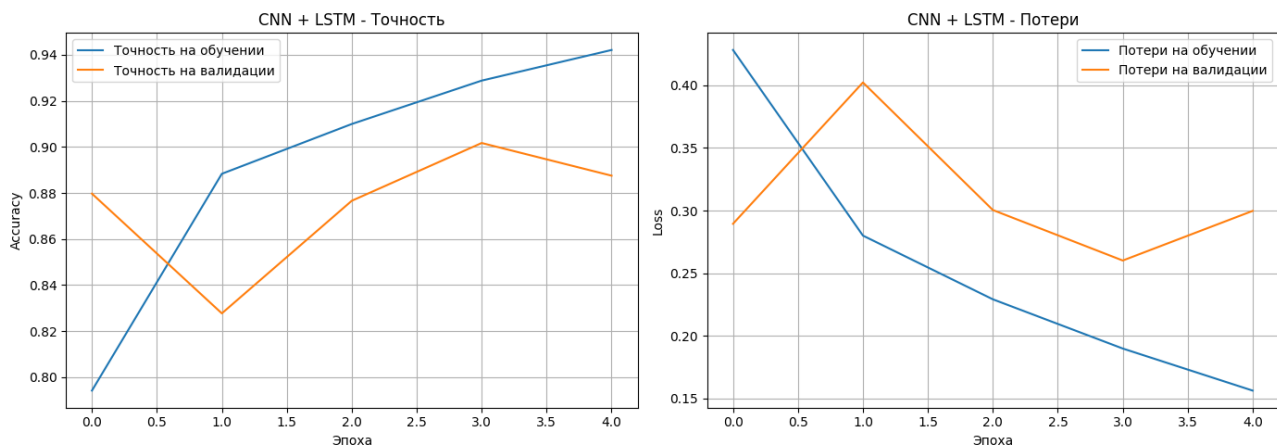


Рисунок 4. Точность и потери в ходе обучения CNN + LSTM.

По графикам можно видеть нестабильное поведение валидационной точности: после первой эпохи (0.8797) она падает до 0.8277 на второй, затем восстанавливается и достигает максимума 0.9017 на четвертой эпохе, после чего слегка снижается. Такие колебания могут быть связаны с особенностями совместного обучения сверточных и рекуррентных слоев, а также с возможным переобучением. Тем не менее, модель показывает высокое качество (0.8875 на финальной эпохе) и является самой быстрой из-за уменьшения размерности после пулинга.

4. Двухслойная LSTM

Архитектура:

- Embedding (10000, 32, input_length=500)
- LSTM (64 нейрона, return_sequences=True, dropout=0.2, recurrent_dropout=0.2)
- LSTM (64 нейрона, dropout=0.2, recurrent_dropout=0.2)
- Dense (1, сигмоид)

Идея заключается в увеличении глубины за счет двух последовательных LSTM-слоев позволяет модели изучать более сложные иерархические зависимости во временной последовательности. Первый слой возвращает всю последовательность скрытых состояний, которые подаются на второй LSTM.

Обучение проводилось на 5 эпохах с размером пакета 64, оптимизатором Adam, функцией потерь бинарная кросс-энтропия. Валидация проводилась на тестовой выборке. Результаты в ходе обучения представлены на рисунке 5.

Точность на тесте составила 0.8904, а время обучения ~826–882 секунд на эпоху (самое долгое, так как два LSTM-слоя сильно увеличивают число параметров и вычислительную сложность).

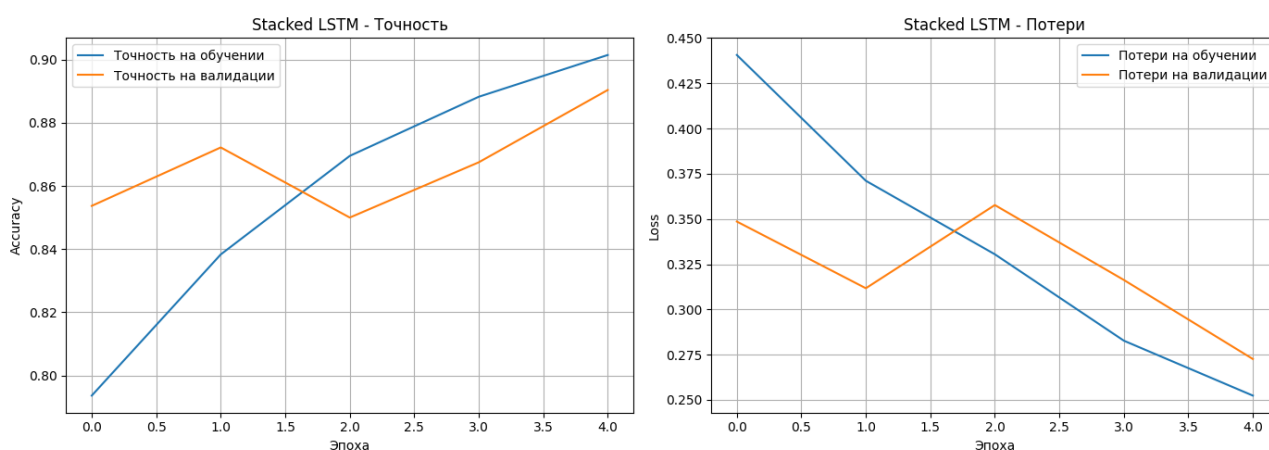


Рисунок 5. Точность и потери в ходе обучения 2х слойной LSTM.

Анализируя динамику обучения, можно отметить следующее:

- На первых двух эпохах точность на обучении и валидации растет синхронно, разрыв минимален.
- На третьей эпохе происходит кратковременное расхождение: точность на обучении продолжает расти (0.8695), а валидационная точность снижается до 0.8500, что является признаком начинающегося переобучения. Потери на валидации также возрастают.
- Однако на четвертой и пятой эпохах разрыв сокращается: к пятой эпохе точность на валидации достигает 0.8904. Это говорит о том, что модель смогла скорректироваться и улучшить обобщающую способность после временной нестабильности.

Такое поведение характерно для глубоких рекуррентных сетей: они могут проходить через локальные минимумы, и даже при кратковременном ухудшении валидационных метрик дальнейшее обучение может привести к улучшению. В данном случае, несмотря на небольшое переобучение в середине процесса, итоговая точность оказалась самой высокой среди всех моделей.

Ансамблирование

Ансамбль построен путем усреднения вероятностей, предсказанных четырьмя обученными моделями, с последующим порогом 0.5 для бинарной классификации. Такой подход часто дает более стабильные результаты, так как компенсирует ошибки отдельных моделей.

Точность ансамбля на тестовой выборке составила 0.8943 (89.43%)

Таким образом, ансамбль превзошел каждую отдельную модель, что подтверждает эффективность комбинирования разных архитектур.

Анализ результатов

Таблица 1. Сравнение моделей

Модель	Точность	Время на эпоху (среднее)	Примечания
Simple LSTM	0.8736	~700	Хорошая базовая модель
LSTM + Dropout	0.8617	~875	Dropout не помог, возможно, из-за слишком высокого коэффициента
CNN + LSTM	0.8875	~360	Быстрая, хорошая точность
Stacked LSTM	0.8904	~850	Лучшая точность, но медленная
Ансамбль	0.8943	—	Наилучший результат

Таким образом можно сделать следующие наблюдения:

- Наиболее быстро обучалась комбинированная модель CNN+LSTM, так как после свертки и пулинга размерность признаков уменьшается, что ускоряет обработку LSTM.
- Самой медленной оказалась Stacked LSTM из-за двух последовательных рекуррентных слоев с большим числом параметров
- Добавление Dropout (0.5) не принесло улучшения – возможно, следовало использовать меньшее значение (например, 0.2–0.3) или увеличить число эпох
- Комбинированная модель показала лучший компромисс между скоростью и качеством
- Ансамблирование дало небольшое, но стабильное повышение точности.

Заявленная в задании цель 97% не была достигнута по следующим причинам:

- Ограниченный объем данных: 40 000 отзывов для обучения – достаточно, но не критически много. Более глубокие модели требовали бы больше данных для обобщения.
- Отсутствие предобученных эмбеддингов: Использовался слой Embedding, обучаемый с нуля. Применение предобученных векторов (GloVe, word2vec) могло бы улучшить качество.
- Недостаточная настройка гиперпараметров: Параметры (размерность эмбеддинга, количество нейронов, dropout, количество эпох) выбирались интуитивно, не проводился систематический поиск.
- Сложность задачи: Хотя задача сентимент-анализа на IMDb считается относительно простой, 97% – это очень высокий порог, приближающийся к человеческому уровню. Обычно лучшие модели на этом датасете достигают 93–94% точности без использования дополнительных данных.

Тестирование на пользовательских текстах

После обучения ансамбля была реализована функция `predict_ensemble`, которая принимает произвольный текст, кодирует его в последовательность индексов (с учетом словаря) и передает всем моделям. Предсказания усредняются, и выдается итоговая метка. Результаты тестирования представлены на рисунке 6.

```
Текст: This movie is great amazing wonderful fantastic.  
Вероятность положительного отзыва: 0.9761  
Результат: ПОЛОЖИТЕЛЬНЫЙ  
Текст: It is terrible movie.  
Вероятность положительного отзыва: 0.3011  
Результат: ОТРИЦАТЕЛЬНЫЙ  
Текст: The movie is terrible.  
Вероятность положительного отзыва: 0.6757  
Результат: ПОЛОЖИТЕЛЬНЫЙ  
Текст: The movie was okay.  
Вероятность положительного отзыва: 0.6372  
Результат: ПОЛОЖИТЕЛЬНЫЙ  
Текст: The acting was bad but the story was good.  
Вероятность положительного отзыва: 0.3615  
Результат: ОТРИЦАТЕЛЬНЫЙ  
Текст: This movie was absolutely fantastic! The acting was superb and the story kept me on the edge of my seat.  
Вероятность положительного отзыва: 0.9257  
Результат: ПОЛОЖИТЕЛЬНЫЙ  
Текст: I hated every minute of it. Boring plot and terrible acting. Complete waste of time.  
Вероятность положительного отзыва: 0.0193  
Результат: ОТРИЦАТЕЛЬНЫЙ  
Текст: It was okay, nothing special but not bad either.  
Вероятность положительного отзыва: 0.1901  
Результат: ОТРИЦАТЕЛЬНЫЙ  
Текст: The actor casting was bad but the plot was well thought out, overall it's worth watching.  
Вероятность положительного отзыва: 0.7867  
Результат: ПОЛОЖИТЕЛЬНЫЙ
```

Рисунок 6. Тестирование на пользовательском тесте.

Результаты показывают, что модель иногда ошибается в интерпретации отрицательных слов и сложных конструкций (например, "not bad"). Это связано с тем, что рекуррентные сети не идеально справляются с отрицаниями и требуют более глубокого понимания контекста.

Выводы.

В ходе лабораторной работы были успешно реализованы и обучены четыре модели рекуррентных нейронных сетей для классификации тональности текстов. Лучшую индивидуальную точность показала двухслойная LSTM (89.04%), а ансамблирование позволило повысить ее до 89.43%.

Были проанализированы различия в скорости обучения и качестве классификации. Установлено, что комбинация сверточных и рекуррентных слоев (CNN+LSTM) дает выигрыш во времени при сохранении высокой точности. Добавление Dropout не всегда полезно, если коэффициент выбран слишком большим.

Достигнутые результаты не дотягивают до 97% ввиду ограниченности используемых данных и архитектур.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab7.ipynb

```
"""# Установка зависимостей
"""
```

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

```
"""# Импорт"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Dense, LSTM, Embedding, Dropout, Conv1D, MaxPooling1D,
GlobalMaxPooling1D
from keras.preprocessing import sequence
from keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
"""# Загрузка данных"""
```

```
def load_data(num_words=10000):
    (train_data, train_targets), (test_data, test_targets) =
imdb.load_data(num_words=num_words)
```

```
    data = np.concatenate((train_data, test_data), axis=0)
    targets = np.concatenate((train_targets, test_targets), axis=0)
```

```
    return data, targets
```

```
num_words = 10000
max_review_length = 500
embedding_dim = 32
```

```
data, targets = load_data(num_words=num_words)
word_index = imdb.get_word_index()
```

```
"""# Предобработка данных"""
```

```
X_train, X_test, y_train, y_test = train_test_split(data, targets, test_size=0.2,
random_state=42)
print(f"Размер обучающей выборки: {len(X_train)}")
print(f"Размер тестовой выборки: {len(X_test)}")
```

```
X_train = sequence.pad_sequences(X_train, maxlen=max_review_length)
X_test = sequence.pad_sequences(X_test, maxlen=max_review_length)
print(f"Форма обучающих данных после паддинга: {X_train.shape}")
print(f"Форма тестовых данных после паддинга: {X_test.shape}")
```

```

"""# Визуализация"""

def plot_history(history, title="Графики обучения"):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    axes[0].plot(history.history['accuracy'], label='Точность на обучении')
    axes[0].plot(history.history['val_accuracy'], label='Точность на валидации')
    axes[0].set_title(f'{title} - Точность')
    axes[0].set_xlabel('Эпоха')
    axes[0].set_ylabel('Accuracy')
    axes[0].legend()
    axes[0].grid(True)

    axes[1].plot(history.history['loss'], label='Потери на обучении')
    axes[1].plot(history.history['val_loss'], label='Потери на валидации')
    axes[1].set_title(f'{title} - Потери')
    axes[1].set_xlabel('Эпоха')
    axes[1].set_ylabel('Loss')
    axes[1].legend()
    axes[1].grid(True)
    plt.tight_layout()
    plt.show()

"""# Простая модель"""

def build_simple_lstm(vocab_size, max_len, embed_dim):
    model = Sequential()
    model.add(Embedding(vocab_size, embed_dim, input_length=max_len))
    model.add(LSTM(100, dropout=0.2, recurrent_dropout=0.2))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

model_lstm = build_simple_lstm(num_words, max_review_length, embedding_dim)
model_lstm.summary()

history_lstm = model_lstm.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=5,
    batch_size=64,
    verbose=1
)

plot_history(history_lstm, "Simple LSTM")

"""# Модель с дополнительным слоем Dropout"""

def build_lstm_dropout(vocab_size, max_len, embed_dim):
    model = Sequential()
    model.add(Embedding(vocab_size, embed_dim, input_length=max_len))
    model.add(Dropout(0.5))

```

```

model.add(LSTM(128, dropout=0.3, recurrent_dropout=0.3))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
return model

model_lstm_drop = build_lstm_dropout(num_words, max_review_length, embedding_dim)
model_lstm_drop.summary()

history_lstm_drop = model_lstm_drop.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=5,
    batch_size=64,
    verbose=1
)

plot_history(history_lstm_drop, "LSTM с Dropout")

"""# Модель из CNN + LSTM"""

def build_cnn_lstm(vocab_size, max_len, embed_dim):
    model = Sequential()
    model.add(Embedding(vocab_size, embed_dim, input_length=max_len))
    model.add(Conv1D(filters=32, kernel_size=3, padding='same', activation='relu'))
    model.add(MaxPooling1D(pool_size=2))
    model.add(LSTM(100, dropout=0.2, recurrent_dropout=0.2))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

model_cnn_lstm = build_cnn_lstm(num_words, max_review_length, embedding_dim)
model_cnn_lstm.summary()

history_cnn_lstm = model_cnn_lstm.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=5,
    batch_size=64,
    verbose=1
)

plot_history(history_cnn_lstm, "CNN + LSTM")

"""# Двухслойная модель LSTM"""

def build_stacked_lstm(vocab_size, max_len, embed_dim):
    model = Sequential()
    model.add(Embedding(vocab_size, embed_dim, input_length=max_len))
    model.add(LSTM(64, return_sequences=True, dropout=0.2, recurrent_dropout=0.2))
    model.add(LSTM(64, dropout=0.2, recurrent_dropout=0.2))
    model.add(Dense(1, activation='sigmoid'))

```

```

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
return model

model_stacked = build_stacked_lstm(num_words, max_review_length, embedding_dim)
model_stacked.summary()

history_stacked = model_stacked.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=5,
    batch_size=64,
    verbose=1
)

plot_history(history_stacked, "Stacked LSTM")

"""# Ансамблирование моделей"""

# Используем обученные модели
models = [model_lstm, model_lstm_drop, model_cnn_lstm, model_stacked]

# Для сравнения, точность каждой отдельной модели:
for i, model in enumerate(models):
    _, acc = model.evaluate(X_test, y_test, verbose=1)
    print(f"Модель {i+1} точность: {acc:.4f}")

# Вычисляем предсказания для тестовой выборки каждой моделью
y_pred_proba = []
for model in models:
    y_pred_proba.append(model.predict(X_test, verbose=1))

# Усредняем вероятности
ensemble_proba = np.mean(y_pred_proba, axis=0)
ensemble_pred = (ensemble_proba > 0.5).astype(int)

# Вычисляем точность ансамбля
ensemble_accuracy = accuracy_score(y_test, ensemble_pred)
print(f"Точность ансамбля (усреднение вероятностей): {ensemble_accuracy:.4f}
({ensemble_accuracy*100:.2f}%)")

"""# Пользовательский текст через ансамблирование"""

def encode_text(text, word_index, num_words=10000):
    words = text.lower().split()
    encoded = []
    for word in words:
        idx = word_index.get(word)
        if idx is not None and idx < num_words:
            encoded.append(idx + 3)
        else:
            encoded.append(2)
    return encoded

```

```

def predict_ensemble(text, models, word_index, max_len=500, num_words=10000):
    encoded = encode_text(text, word_index, num_words)
    padded = sequence.pad_sequences([encoded], maxlen=max_len)
    preds = []
    for model in models:
        prob = model.predict(padded, verbose=0)[0][0]
        preds.append(prob)
    ensemble_prob = np.mean(preds)
    return ensemble_prob, "ПОЛОЖИТЕЛЬНЫЙ" if ensemble_prob > 0.5 else
"ОТРИЦАТЕЛЬНЫЙ"

test_texts = [
    "This movie is great amazing wonderful fantastic.",
    "It is terrible movie.",
    "The movie is terrible.",
    "The movie was okay.",
    "The acting was bad but the story was good.",

    "This movie was absolutely fantastic! The acting was superb and the story kept me on the
edge of my seat.",
    "I hated every minute of it. Boring plot and terrible acting. Complete waste of time.",
    "It was okay, nothing special but not bad either.",
    "The actor casting was bad but the plot was well thought out, overall it's worth watching."
]
for text in test_texts:
    prob, sentiment = predict_ensemble(text, models, word_index,
max_len=max_review_length)
    print(f"Текст: {text}")
    print(f"Вероятность положительного отзыва: {prob:.4f}")
    print(f"Результат: {sentiment}")

```